CppHDL Specification

Mike Reznikov

2026-01-10

This document is currently in **draft** status.
Content may change significantly before final approval.

Contents

Mapping of SystemVerilog Expressions to C++

CppHDL code should be read as a direct C++ mapping of synthesizable SystemVerilog RTL. Continuous assignments and module port connections are written in the `_assign()` section. This section runs only once, before the work cycle starts, and binds C++ lambdas that are used later during simulation and SystemVerilog generation. The `_ASSIGNxxx()` macros are only allowed in `_assign()`.

SystemVerilog:

```
    output wire out
);
assign out = a + b;
assign valid_out = valid;
```

CppHDL:

```
_PORT(u<32>) out = _ASSIGN(a_in() + b_in());

void _assign()
{
    valid_out = _ASSIGN_REG(valid_reg);
}
```

Use `_ASSIGN(expr)` for expressions. Use `_ASSIGN_REG(reg_or_signal)` for direct storage bindings such as registers, logic values, memories, or ports whose final object reference is enough. Use `_ASSIGN_COMB(comb_func())` when assigning the result of a CppHDL combinational function. Even though `_ASSIGN_COMB()` captures the returned object by reference, the `comb_func()` call itself is still executed on demand when the port value is read. For loop-indexed assignments use `_ASSIGN_I`, `_ASSIGN_REG_I`, `_ASSIGN_COMB_I`, or the indexed forms such as `_ASSIGN_INDEXED((i,j,k), expr)` and `_ASSIGN_REG_INDEXED((i,j,k), object[i][j][k])`.

All SystemVerilog `always_ff` blocks for one module map into one CppHDL `_work(bool reset)` method. `_work()` computes next register values. It may contain the logic that would be split across several `always_ff` blocks in SystemVerilog.

SystemVerilog:

```
always_ff @(posedge clk) begin
    if (reset) count <= '0;
    else if (enable) count <= count + 1;
end

always_ff @(posedge clk) begin
```

```
    if (reset) valid <= 1'b0;
    else valid <= enable_in;
end
```

CppHDL:

```
reg<u<8>> count_reg;
reg<u1> valid_reg;

void _work(bool reset)
{
    if (reset) {
        count_reg.clr();
        valid_reg.clr();
        return;
    }

    if (enable_in()) {
        count_reg._next = count_reg + u<8>(1);
    }
    valid_reg._next = enable_in();
}
```

SystemVerilog `always_comb` blocks map to CppHDL combinational functions, usually named `*_comb_func()`. They calculate temporary combinational values from current inputs and current register values. The usual style is to store the result in a member variable and return it by reference, as shown in the root examples.

SystemVerilog:

```
always_comb begin
    hit = valid && tag == req_tag;
    read_data = hit ? line[word] : '0;
end
```

CppHDL:

```
bool hit_comb_func()
{
    return valid_reg && tag_reg == req_tag_in();
}

u<32> read_data_comb_func()
{
    return hit_comb_func() ? line_reg[word_in()] : u<32>(0);
}
```

CppHDL commits registers and memories in the mandatory `_strobe()` method. `_strobe()` is executed recursively for each module at the end of each clock evaluation. Register `.strobe()` calls and memory `.apply()` calls are only allowed in `_strobe()`, not in `_assign()`, `_work()`, or comb functions.

```
void _strobe()
{
    count_reg.strobe();
    valid_reg.strobe();
    member._strobe();
}
```

Introduction

CppHDL is a C++ hardware definition language extension for digital integrated circuit development, designed for two purposes:

1. Building a full cycle of digital RTL development and testing using the C++ language
2. Allowing extremely fast simulation of RTL defined with blocking assignments

In all operations CppHDL works as a reflection of the SystemVerilog model, which means that, at every stage, a 100% register-to-register copy of any CppHDL code exists in the SystemVerilog domain. This live CppHDL to SystemVerilog conversion makes it possible to

- Connect CppHDL teams to classical verification and testing teams
- Deliver SV RTL to fabrication processes and tools or third-party companies

The main benefits of using C++ for RTL development are replacing slow **simulation** with compilation and execution that can be up to 100 times faster, while using a modern language that is accessible to more developers. The following properties of the C++ language provide a strong foundation for the RTL development process:

- Ability to use many professional IDEs and tools for development and debugging, including support for large project management
- C++ is one of the most popular and powerful programming languages in the world, with an extremely wide community
- C++ is precisely understood language by modern AI models and extremely useful in both rapid hardware verification and developmet
- CppHDL makes many of C++ developers accessible for chipmaking industry
- C++ is extremely fast in compilation and execution
- It is free and does not require paying for instances

RTL modeling using CppHDL includes verification and testing, providing the power and speed of the C++ language for modeling digital signaling and digital system interaction. It makes the verification process simpler, more flexible, faster, and purely programmatic.

CppHDL generates pure SystemVerilog output that is mostly a reflection of the C++ source, providing line-to-line visibility. It is even possible to apply patches synchronously to both RTL representations during finalization. Generated SystemVerilog files can be frozen at any moment and used as the main source code for the next stage of the ASIC/FPGA production process.

Who is CppHDL for

- IC development AI Agent's shepherds (agent does 100 times more iterations per day and understands C++ language better)
- Digital IC development teams (ASIC, IP, libraries, FPGA) - for faster development and testing of complex digital designs, free of charge
- Digital IC developers - to use modern C++ environments and smart IDEs, powerful C++ debug tools, static analysis, and linting
- Software developers who want to deliver hardware and use a powerful modern language with OOP, templates, abstraction, recursion, etc.
- CAD/tool developers (especially AI-coding/training) - 100 times more work cycles per day, with C++ being better learned by GPTs
- Talent seekers - involving speakers of a popular programming language in a variety of modern projects

What is the difference from other C++ to Verilog products

- CppHDL is not HLS; it is a representation of SystemVerilog, register to register and clock to clock, completely repeating model behavior line by line
- A CppHDL module is a simple single-process activity without waits for synchronization, notifications, `yield()`, streams, or cooperative multitasking

Limitations

- CppHDL supports only digital design components written using blocking assignments
- Currently no multiclock or CDC is supported. Each clock domain should be developed separately
- Timing- or power-critical sections should be isolated at the architectural level

CppHDL is intended to bring ease and speed to the development of various digital circuits such as controllers, multiplexers, cache and memory functions, mathematical functions, digital data processing, transmitting circuits, etc.

Requirements

CppHDL is delivered in two parts:

- C++ headers that contain definitions of CppHDL datatypes
- A conversion/linter tool that provides CppHDL to SystemVerilog conversion

Since CppHDL works as a reflection of the SystemVerilog RTL model, a strong understanding of SystemVerilog modeling techniques is required, in particular:

- Synchronous logic digital circuits
- Blocking and non-blocking assignments
- Combinational logic digital circuits
- Structures, packages, packed arrays, and unpacked arrays

CppHDL syntax

A CppHDL source file can be written as an `.h` header or a `.cpp` object file. Each header file should describe a module or auxiliary information such as datatypes, constants, interfaces, inline function definitions, etc.

CppHDL provides the ability to use C++ structures and custom datatypes in order to build a comfortable and consistent ecosystem for project development. All types and constants are translated into SystemVerilog datatypes during the conversion process.

The following example shows appropriate usage of C++ defines and structures in CppHDL.

```
#pragma once

#include "cpphdl.h"
#include "PrjConfig.h"

using namespace cpphdl;

#define    CMD_IDLE    0
#define    CMD_RESET   1
#define    CMD_CONFIG  2

struct CmdConfig
{
    unsigned cmd_id;
    unsigned units:6;
    unsigned flags:2;
    unsigned address;
}__PACKED;
static_assert (sizeof(CmdConfig) == 4, "struct CmdConfig size is not correct");
```

Module description format

Module definition is a C++ class with cpphdl::Module base, which includes:

1. Optional private zone with nested modules
2. Public zone with I/O ports definitions and *_assign*() function body
3. Optional private zone for registers and variables
4. Public zone with *_work(reset)*, *_strobe*(), *_work_neg(reset)*, *_strobe_neg*() and combinational functions bodies

In the following block of code a simple FIFO model RTL shown as a basic CppHDL example:

```cpp
#pragma once

#include "cpphdl.h"
#include "Memory.h"
#include <print>

using namespace cpphdl;

template<size_t FIFO_WIDTH_BYTES, size_t FIFO_DEPTH, bool SHOWAHEAD = true>
class Fifo : public Module
{
    Memory<FIFO_WIDTH_BYTES,FIFO_DEPTH,SHOWAHEAD> mem;

public:
    _PORT(bool)                         write_in;
    _PORT(logic<FIFO_WIDTH_BYTES*8>)    write_data_in;

    _PORT(bool)                         read_in;
    _PORT(logic<FIFO_WIDTH_BYTES*8>)    read_data_out  = mem.read_data_out;

    _PORT(bool)                         empty_out      = _ASSIGN_REG( empty_comb_func()
    ↪  );
    _PORT(bool)                         full_out       = _ASSIGN_REG( full_comb_func() );
    _PORT(bool)                         clear_in       = _ASSIGN( false );
    _PORT(bool)                         afull_out      = _ASSIGN_REG( afull_reg );

    bool                        debugen_in;

private:
    reg<u<clog2(FIFO_DEPTH)>> wp_reg;
    reg<u<clog2(FIFO_DEPTH)>> rp_reg;
    reg<u1> full_reg;
    reg<u1> afull_reg;

public:

    void _assign()
    {
        mem.write_data_in = write_data_in;
        mem.write_data_in = write_data_in;
        mem.write_in      = write_in;
        mem.write_mask_in = _ASSIGN( 0xFFFFFFFFFFFFFFFFULL );
        mem.write_addr_in = _ASSIGN_REG( wp_reg );
        mem.read_in       = read_in;
        mem.read_addr_in  = _ASSIGN_REG( rp_reg );
        mem.__inst_name = __inst_name + "/mem";
        mem.debugen_in  = debugen_in;
        mem._assign();
    }

    u1 full_comb;
    bool& full_comb_func()
    {
```

```
53          return full_comb = (wp_reg == rp_reg) && full_reg;
54      }
55
56      u1 empty_comb;
57      bool& empty_comb_func()
58      {
59          return empty_comb = (wp_reg == rp_reg) && !full_reg;
60      }
61
62      void _work(bool reset)
63      {
64          mem._work(reset);
65
66          if (debugen_in) {
67              std::print("{:s}: input: ({})({}), output: ({})({}), wp_reg: {}, rp_reg: {},
                    ↪ full: {}, empty: {}, reset: {}\n", __inst_name,
68                  (int)write_in(), write_data_in(), (int)read_in(), read_data_out(),
                        ↪ wp_reg, rp_reg, (int)full_out(), (int)empty_out(), reset);
69          }
70
71          if (reset) {
72              wp_reg.clr();
73              rp_reg.clr();
74              full_reg.clr();
75              afull_reg.clr();
76              return;
77          }
78
79          if (write_in()) {
80
81              if (full_comb_func() && !read_in()) {
82                  std::print("{:s}: writing to a full fifo\n", __inst_name);
83                  exit(1);
84              }
85              if (!full_comb_func()  read_in()) {
86                  wp_reg._next = wp_reg + 1;
87              }
88              if (wp_reg._next == rp_reg) {
89                  full_reg._next = 1;
90              }
91          }
92
93          if (read_in()) {
94
95              if (empty_comb_func()) {
96                  std::print("{:s}: reading from an empty fifo\n", __inst_name);
97                  exit(1);
98              }
99              if (!empty_comb_func()) {
100                 rp_reg._next = rp_reg + 1;
101             }
102             if (!write_in()) {
103                 full_reg._next = 0;
104             }
```

```
105         }
106
107         if (clear_in()) {
108             wp_reg._next = 0;
109             rp_reg._next = 0;
110             full_reg._next = 0;
111         }
112
113         afull_reg._next = full_reg  (wp_reg >= rp_reg ? wp_reg - rp_reg : FIFO_DEPTH -
            ↪  rp_reg + wp_reg) >= FIFO_DEPTH/2;
114     }
115
116     void _strobe()
117     {
118         mem._strobe();
119         wp_reg.strobe();
120         rp_reg.strobe();
121         full_reg.strobe();
122         afull_reg.strobe();
123     }
124 };
```

- A module class definition can use template parameters
- Built-in C++ types such as `bool`, `unsigned`, `unsigned long`, etc. are allowed in all places except `reg<>`
- Each of the *_assign*(), *_work*(), and *_strobe*() functions should call the corresponding functions of nested modules
- Only the *reg_name._next* value can be changed outside reset. Both `reg` and *reg._next* values can be used on the right side of expressions
- During reset, *.clr*() and *.set(val)* methods are used to set both current and next register values
- CppHDL replaces *[f]printf*(), *std::print*, *$write*(), and *exit*() functions with their SV equivalents, and parameters are converted appropriately

Input/output ports

All ports are of type *std::function<**data_type()**>* in CppHDL. This allows instant recalculation of complete combinational function chains.

- Macro *_PORT(**data_type**)* allows simple port declaration.
- Macro *_ASSIGN(**any_cpp_expression**)* represents a Verilog assign expression. The return type can be cast, and the compiler checks the lambda return type.

- Macro *_ASSIGN_REG(`member_or_signal`)* is a fast replacement for *_ASSIGN*() when the value is a persistent variable in the class.
- Macro *_ASSIGN_COMB(`comb_func()`)* is used for combinational functions that return a reference to a persistent result variable.

The following naming convention is used for input/output ports (use `_in` ending or `_out` ending):

- `port_in` or `obj_port_in` - generic input port name
- `port_out` or `obj_port_out` - generic output port name
- or longer name, ending with *_in* or *_out*

It is recommended to initialize all output ports directly in the class description when possible. In more complex situations, it is recommended to initialize output ports in a special *_assign*() function. Input ports can be assigned the *_ASSIGN(0)* value to emulate Verilog unassigned input behavior.

NOTE! To build a complex bus interface between a CppHDL module and a third-party SV module, use packed *structs* to achieve proper `<8`bit fields packing.

Clock and reset

Currently, only one clock is used, named *clk*. Reset is the main *reset* parameter of the work function. It is possible to define any synchronous reset sequence. Asynchronous reset is not supported yet.

Variables list

Variables are module class members and can be of one of 3 types:

- **registers**
- **combinational**
- **temporary**

Registers are of type **`reg<TYPE>`** and contain value, updated on strobing clock edge. To access next value of a register the *reg_name._next* property is used. It is recommended to give register names with a *reg* suffix in case when register is used as output port or in parent modules.

- Registers can carry structs, arrays, and single values.
- Combinational variable can be of any type.
- Variables can be declared inside methods/functions, but declarations are allowed only at the very beginning of the method/function, as in C.

Connect method

The *_assign*() method is used to assign nested instance inputs to data sources in the module and back again.

Work method

The work method can make changes to registers and temporary variables. Only *._next* value of registers should be changed directly.

- Work method can call other methods to make code well-structured.
- Methods with return values become SystemVerilog functions; methods with `void` return become Verilog tasks.
- It is possible to change registers only from void methods.
- Methods can take references to registers as parameters.

Strobe method

The strobe function should contain all registers of the module and call `.strobe()` functions for them. Also, `_strobe()` should be called for each nested instance of the class. Forgotten registers will be reported by *cpphdl* tool.

Comb methods

Combinational methods represent Verilog combinational logic functions. All combinational methods should comply with the following requirements:

- The name of the function should contain *_comb_func*() suffix
- A corresponding variable should be defined in the module class: *var_name_comb*
- The combinational function should calculate and assign a value to the *var_name_comb* variable, then return a reference to it
- **NOTE!** The global variable *sys_clock* is used to cache and update all combinational values only after a clock edge switch. Use of this variable is mandatory (see examples)

It will be converted to a corresponding Verilog variable and `always @(*)` block during conversion.

It is important to avoid loops in combinational function call chains.

Data types

To repeat SystemVerilog behavior, CppHDL implements a number of basic data types, each corresponding to a specific SystemVerilog datatype. Currently, the list of CppHDL datatypes includes:

- *logic<WIDTH>* - any width variable, optimized for bit-access
- *u<WIDTH>*, u1, u8, u16, u32, u64 - unsigned variables
- *s<WIDTH>*, s1, s8, s16, s32, s64 - signed variables (reserved but not implemented because of lack of demand and examples)
- *reg<TYPE>* - register definition, works only with CppHDL types or any structs
- *array<TYPE,SIZE>* - variable optimized for large array access and element changes
- *memory<TYPE,SIZE>* - special registered container implementing optimal memory access with strobing
- *Cat{...}* - concatenation helper that maps to SystemVerilog `{...}` expressions

logic<WIDTH>

logic<> is the basic type of the CppHDL toolchain, representing the SystemVerilog `logic` type. It is universal, can be of any width, and can be used as a standalone variable or inside a reg<> construction.

Example usage as a variable:

```
logic<MEM_WIDTH_BYTES*8> data_out_comb;
```

Example usage as a port:

```
_PORT(logic<MEM_WIDTH_BYTES*8>) data_in = nullptr;
```

The logic<> type provides read/write access to individual bits using *operator[]* and to partial bitmaps using the *.bits(hi,lo)* method:

```
buffer1_byteenable._next[addr_sub+i] = 1;
host_addr.bits(39,32) = s_writedata_in() >> 32;
```

The *.bits(hi,lo)* arguments can be expressions, so indexed slices such as `bits(word * 16 + 15, word * 16)` are supported and converted to SystemVerilog indexed part-selects. The slice width must be statically implied by the expression shape: use the same base expression on both sides plus a constant width, keep `hi >= lo`, and keep the selected range inside the `logic<>` width. Dynamic indexed slices are intended for contiguous read/write fields, not for arbitrary variable-width ranges.

Examples from `tests/datatypes/LogicBitsIndexing.cpp`:

```
logic<128> source_comb;
logic<16> direct_comb;
logic<8> byte_comb;

direct_comb = source_comb.bits(word * 16 + 15, word * 16);
byte_comb = source_comb.bits(word * 8 + 7, word * 8);
```

Writing indexed slices is supported as well:

```
logic<128> edited_comb;

edited_comb.bits(word * 16 + 15, word * 16) =
    logic<16>((uint64_t)seed_in() ^ 0x55aa);

edited_comb.bits((word + 1) * 16 + 15, (word + 1) * 16) =
    logic<16>((uint64_t)seed_in() ^ 0xaa55);

edited_comb.bits(word * 8 + 7, word * 8) =
    logic<8>((uint64_t)seed_in() ^ 0x5a);
```

u<WIDTH>

u<> is a basic unsigned value of variable size that supports all math operators and can be cast to a logic<> variable. Although *u*<> can be of any size, it supports a maximum of 64-bit math. Example usage of *u*<>:

```
u<STEPS_SIZE> cmd_steps;
```

Cat{...}

Cat{...} builds a packed concatenation from CppHDL values and maps directly to a SystemVerilog concatenation expression. Arguments are placed from high bits to low bits in the same order as SystemVerilog `{a, b, c}`. The result width is inferred from the argument types. Supported operands include `logic<WIDTH>`, `u<WIDTH>`, fixed unsigned aliases such as `u8` and `u32`, and `reg<T>` values whose stored type is supported.

Example assignment to a port:

```
u<4> byteenable;
u<5> address;
u<8> data;

void _assign()
```

```
{
    buffer.valid_in = _ASSIGN(Cat{byteenable, address, data});
}
```

This is emitted as a SystemVerilog concatenation:

```
assign buffer__valid_in = {byteenable, address, data};
```

The same helper can be used for register next-state assignment:

```
reg<logic<17>> packet_reg;
reg<u<4>> reg_byteenable;
reg<u<5>> reg_address;
reg<u<8>> reg_data;

void _work(bool reset)
{
    packet_reg._next = Cat{reg_byteenable, reg_address, reg_data};
}
```

The destination type must be wide enough for the concatenation result. Width mismatches are checked by the C++ type system and by the generated SystemVerilog tools.

u1, u8, u16, u32, u64

u1, *u8*, *u16*, *u32*, and *u64* are aliases for *u<1>*, *u<8>*, *u<16>*, *u<32>*, and *u<64>*, respectively.

reg<TYPE>

The reg<> template is intended to make a variable a register. It adds the .*_next* property, which is changed in a *_work*() function, as well as the ._strobe() method, which synchronizes the current value with the next value. It should not be used as a port definition, but it can provide data to a port. Examples of reg<> usage are provided below:

```
reg<State> state;
reg<u16> size;
reg<array<u8,WIDTH/8>> buffer1;
reg<logic<WIDTH/8>> buffer1_byteenable;
```

```
state_struct._next.steps = state_struct.steps - 1;
if (state_struct.steps == 0) {
    state_struct._next.steps = 255;
}

size._next = 0xFFFF;

buffer1._next |= buffer1_precalc;

buffer1_byteenable._next[i] = buffer2_byteenable[i];

mask._next.bits((i+1)*32-1,i*32) = 0;
```

array<SIZE>

The array<> type is used for storing a vector of similar types. It can be used with the reg<> template.

```
_PORT(array<u8,WIDTH/8>) avmm_writedata_out = _ASSIGN_REG( buffer1 );
```

memory<TYPE,SIZE>

The *memory*<> type is developed for optimal access performance to registered memory, with the ability to change one word per clock cycle. It cannot be used as a port. It uses the *apply*() method for strobing data. The following example shows how memory<> should be used to organize simple memory with one read and one write port.

```
#pragma once

#include "cpphdl.h"
#include <print>

using namespace cpphdl;

template<size_t MEM_WIDTH_BYTES, size_t MEM_DEPTH, bool SHOWAHEAD = true>
class Memory : public Module
{
    reg<logic<MEM_WIDTH_BYTES*8>> data_out_reg;
    memory<u8,MEM_WIDTH_BYTES,MEM_DEPTH> buffer;

    size_t i;

public:
    _PORT(u<clog2(MEM_DEPTH)>)       write_addr_in;
    _PORT(bool)                      write_in;
    _PORT(logic<MEM_WIDTH_BYTES*8>)  write_data_in;
```

```
20      _PORT(logic<MEM_WIDTH_BYTES>)    write_mask_in;
21
22      _PORT(u<clog2(MEM_DEPTH)>)       read_addr_in;
23      _PORT(bool)                      read_in;
24      _PORT(logic<MEM_WIDTH_BYTES*8>)  read_data_out = _ASSIGN_REG( data_out_comb_func() );
25
26      bool                        debugen_in;
27
28      void _assign() {}
29
30      logic<MEM_WIDTH_BYTES*8> data_out_comb;
31      logic<MEM_WIDTH_BYTES*8>& data_out_comb_func()
32      {
33          if (SHOWAHEAD) {
34              data_out_comb = buffer[read_addr_in()];
35          }
36          else {
37              data_out_comb = data_out_reg;
38          }
39          return data_out_comb;
40      }
41
42      logic<MEM_WIDTH_BYTES*8> mask;
43
44      void _work(bool reset)
45      {
46          if (write_in()) {
47              mask = 0;
48              for (i=0; i < MEM_WIDTH_BYTES; ++i) {
49                  mask.bits((i+1)*8-1,i*8) = write_mask_in()[i] ? 0xFF : 0 ;
50              }
51              buffer[write_addr_in()] = (buffer[write_addr_in()]&~mask) |
                ↪  (write_data_in()&mask);
52          }
53
54          if (!SHOWAHEAD) {
55              data_out_reg._next = buffer[read_addr_in()];
56          }
57
58          if (debugen_in) {
59              std::print("{:s}: input: ({})({}@{}({}), output: ({})({}@{}\n", __inst_name,
60                  (int)write_in(), write_data_in(), write_addr_in(), write_mask_in(),
61                  (int)read_in(), read_data_out(), read_addr_in());
62          }
63      }
64
65      void _strobe()
66      {
67          buffer.apply();
68          data_out_reg.strobe();
69      }
70  };
```

Interfaces

Interfaces are special structures which consist of ports of any direction. There is no need for separate "driver" and "responder" interface types: both sides use the same `Interface` type, and the port suffix (`_in` or `_out`) describes signal direction relative to each module. For example, `source_out.valid_in` and `sink_in.valid_in` are both the valid signal driven by the source and consumed by the sink. During SystemVerilog conversion these names are flattened, for example `source_out.valid_in` becomes `source_out__valid_out` on the driver module.

```
template<size_t DATAWIDTH>
struct DataValidReady
{
    bool valid;
    bool ready;
    logic<DATAWIDTH> data;
};

template<size_t DATAWIDTH>
struct DataValidReadyIf : public Interface
{
    _PORT(bool) valid_in;
    _PORT(bool) ready_out;
    _PORT(logic<DATAWIDTH>) data_in;
};

template<size_t DATAWIDTH>
class VRDriver : public Module
{
public:
    DataValidReadyIf<DATAWIDTH> source_out;

private:
    reg<u1> valid_reg;
    reg<logic<DATAWIDTH>> data_reg;

public:
    void _assign()
    {
        source_out.valid_in = _ASSIGN_REG(valid_reg);
        source_out.data_in = _ASSIGN_REG(data_reg);
    }

    void _work(bool reset)
    {
        if (reset) {
            valid_reg.clr();
            data_reg.clr();
            return;
        }

```

```
        valid_reg._next = 1;
        if (!valid_reg  source_out.ready_out()) {
            data_reg._next = data_reg + logic<DATAWIDTH>(1);
        }
    }

    void _strobe()
    {
        valid_reg.strobe();
        data_reg.strobe();
    }
};

template<size_t DATAWIDTH>
class VRResponder : public Module
{
public:
    DataValidReadyIf<DATAWIDTH> sink_in;

private:
    reg<u1> ready_reg;
    reg<logic<DATAWIDTH>> last_data_reg;

public:
    void _assign()
    {
        sink_in.ready_out = _ASSIGN_REG(ready_reg);
    }

    void _work(bool reset)
    {
        if (reset) {
            ready_reg.clr();
            last_data_reg.clr();
            return;
        }

        ready_reg._next = 1;
        if (sink_in.valid_in() && sink_in.ready_out()) {
            last_data_reg._next = sink_in.data_in();
        }
    }

    void _strobe()
    {
        ready_reg.strobe();
        last_data_reg.strobe();
    }
};

class TestValidReady : public Module
{
    VRDriver<32> driver;
    VRResponder<32> responder;
```

```
96
97  public:
98      void _assign()
99      {
100         driver.__inst_name = __inst_name + "/driver";
101         responder.__inst_name = __inst_name + "/responder";
102         assignIf(driver, responder, driver.source_out, responder.sink_in);
103     }
104 };
```

`assignIf(modA, modB, ifA, ifB)` connects two interface instances and calls the modules' `_assign()` methods in the order needed for bidirectional interface signals. This is important for interfaces such as valid-ready, where one module drives `valid` and `data`, while the other drives `ready`. Use it from a parent module or test wrapper after setting instance names. Check `examples/axi/Axi4MuxFromSlave.cpp`, `examples/axi/Axi4MuxToMaster.cpp`, and `tests/interface/ValidReady.cpp` for larger examples.

CppHDL SV Conversion tool

The main purposes of *cpphdl* tool is to

- Provide conversion of CppHDL code to SystemVerilog models
- Check dependencies in combinational chains and forgotten strobe calls in CppHDL source code

Structures

- Each structure or union is converted into SystemVerilog package in a separate file
- The order of fields is reversed due to differences between C++ and SystemVerilog
- Anonymous structures and unions declared inside other structures get 'anon' name substitution
- CppHDL aligns non-bitfield structure members to byte boundaries and emits hidden `_alignN` fields when padding is needed
- Packed unions use the largest branch size; smaller struct/union branches get hidden `_padN` fields so all union alternatives have the same packed width
- `cpphdl::array<T,N>` fields are emitted as packed SystemVerilog arrays inside the generated struct package

Example from `tests/structs/ArrayInStruct.cpp`:

```
1 struct ArrayPayload
2 {
3     unsigned prefix:4;
```

```
    array<u8, 3> bytes;
    unsigned mid:3;
    array<u16, 1> halfs;
    unsigned tail:5;
} __PACKED;
```

Generated SystemVerilog:

```
package ArrayPayload_pkg;

typedef struct packed {
    logic[3-1:0] _align0;
    logic[5-1:0] tail;
    logic[1-1:0][16-1:0] halfs;
    logic[5-1:0] _align2;
    logic[3-1:0] mid;
    logic[3-1:0][8-1:0] bytes;
    logic[4-1:0] _align1;
    logic[4-1:0] prefix;
} ArrayPayload;

endpackage
```

Example from `tests/structs/StructAlignment.cpp`:

```
struct TinyBits
{
    unsigned a:1;
    unsigned b:2;
    unsigned c:3;
} __PACKED;

struct MixedBits
{
    unsigned flag:1;
    unsigned code:4;
    u<3> state;
    unsigned tail:2;
} __PACKED;

struct OuterBits
{
    unsigned head:3;
    TinyBits tiny;
    unsigned mid:5;
    MixedBits mixed;
    u<4> nibble;
    unsigned last:1;
} __PACKED;
```

Generated SystemVerilog:

```
package OuterBits_pkg;
import TinyBits_pkg::*;
import MixedBits_pkg::*;

typedef struct packed {
    logic[7-1:0] _align0;
    logic[1-1:0] last;
    logic[4-1:0] _align3;
    logic[4-1:0] nibble;
    MixedBits mixed;
    logic[3-1:0] _align2;
    logic[5-1:0] mid;
    TinyBits tiny;
    logic[5-1:0] _align1;
    logic[3-1:0] head;
} OuterBits;

endpackage
```

Packed union branches are also normalized to a common size:

```
struct UnionStruct
{
    unsigned sa:4;
    u<6> sb;
    unsigned sc:1;
} __PACKED;

union UnionWithStruct
{
    struct {
        unsigned us0:2;
        UnionStruct nested;
        unsigned us1:3;
    } __PACKED branch;
    struct {
        u<11> other0;
        unsigned other1:5;
    } __PACKED other;
} __PACKED;
```

Generated SystemVerilog:

```
package UnionWithStruct_pkg;

typedef union packed {
```

```
    struct packed {
        logic[24-1:0] _pad1;
        logic[5-1:0] other1;
        logic[11-1:0] other0;
    } other;
    struct packed {
        logic[5-1:0] _align0;
        logic[3-1:0] us1;
        UnionStruct nested;
        logic[6-1:0] _align1;
        logic[2-1:0] us0;
    } branch;
} UnionWithStruct;

endpackage
```

Templates

- During conversion, cpphdl uses `Module` class template parameters as SystemVerilog module parameters if they are numerical
- cpphdl creates a separate SV module for each instantiated combination of data types used as template parameters

References

- All references are removed during SV conversion. References should be used in C++ when necessary and when they improve performance.

Syntax

The *generated* folder is created after a *cpphdl* call and contains `.sv` files. The syntax of the *cpphdl* tool is the following:

```
cpphdl <source.h> <source.cpp> ... [compilation parameters]
```

The cpphdl tool is based on llvm clang and supports all usual C++ command line parameters.

Annotations

CPPHDL_REPLACEMENT

- `[[clang::annotate("CPPHDL_REPLACEMENT=...;")]]` can be attached to a `cpphdl::Module` class.
- During conversion cpphdl stores the text after `CPPHDL_REPLACEMENT=` in `Module::replacement`; a trailing metadata `;` is stripped.
- When project generation sees replacement text, it writes that text directly to the module `.sv` file.
- Normal import, port, register, method, and module body generation is skipped for that module.

Example from `tests/format/AnnotateReplacement.cpp`:

```
class [[clang::annotate(
    "CPPHDL_REPLACEMENT="
    "`default_nettype none\n"
    "\n"
    "module AnnotateReplacement (\n"
    "    input wire clk\n"
    ",   input wire reset\n"
    ",   input wire[8-1:0] value_in\n"
    ",   output wire[8-1:0] value_out\n"
    ");\n"
    "    assign value_out = value_in ^ 8'hA5;\n"
    "endmodule\n"
    ";"
)]] AnnotateReplacement : public Module
{
public:
    _PORT(u<8>) value_in;
    _PORT(u<8>) value_out = _ASSIGN_REG(value_comb_func());

private:
    u<8> value_comb;

    u<8>& value_comb_func()
    {
        return value_comb = value_in() ^ u<8>(0xa5);
    }

public:
    void _work(bool reset) {}
    void _strobe() {}
    void _assign() {}
};
```